HubLink GUI - Class Diagram

Frontend
Components
«React.FC» App
State
-tier: 'desktop' | 'mobile'
-sidebarOpen: boolean
Layout Shell
~ThemeProvider
~LanguageProvider
~AuthProvider
~HealthProvider
~SettingsProvider
~ChatProvider
«React.FC» ImageWithFallback
-didError: boolean
-handleError(): void
«React.FC» ChatPanel
State
-input: string
-topicEntityId: string
-isSettingsOpen: boolean
Methods
-handleSubmit(e: FormEvent): void
-scrollToBottom(): void
Renders
~MessageList
~SettingsModal
~InputForm
«React.FC» NodeGraphPanel
State
-hoveredNodeId: string | null
-clickedEdge: {from, to, relation} | null
-maxDepth: number
-nodePositions: Map<string, {x, y}>
Refs
-canvasRef: RefObject<HTMLCanvasElement>
Methods
-calculateHierarchicalLayout(): void
-drawGraph(): void
-filterNodesByDepth(nodes): Node[]
«React.FC» Sidebar
Props
+isDrawerMode: boolean
State
-collapsed: boolean
-sidebarSearch: string
Renders
~NewChatButton
~ConversationList
~BackendStatusIndicator
renders
renders
renders
provides
provides
provides
provides
provides
provides
useChat()
useChat()
useHealth()
useChat()
useSettings()
useSettings()
reads settings
Contexts
«Context» ChatContext
State
+messages: Message[]
+conversations: Conversation[]
+currentConversationId: string | null
+selectedMessageId: string | null
+isLoading: boolean
+isConversationsLoading: boolean
+displayNodes: Node[]
+selectedGraphMaxDepth: number | null
Actions
+sendMessage(content: string, topicEntityId?: string): Promise<void>
+clearChat(): Promise<void>
+selectMessage(messageId: string): void
+createConversation(): Promise<void>
+switchConversation(id: string): Promise<void>
+deleteConversation(id: string): Promise<void>
«Context» HealthContext
State
+backendStatus: 'checking'|'hublink'|'initializing'|'mock'|'offline'
«Context» LanguageContext
«Context» AuthContext
«Context» ThemeContext
«Context» SettingsContext
State
+retrievalMode: RetrievalMode
+llmModel: string
+numberOfHubs: number
Actions
+setRetrievalMode(mode): void
+setLlmModel(model): void
+setNumberOfHubs(hubs): void
calls API
calls API
Types
Message
+id: string
+role: "user" | "assistant"
+content: string
+nodes?: Node[]
+graphMaxDepth?: number
Conversation
+id: string
+title: string
+createdAt: string
+updatedAt: string
RetrievalMode
direct
graph
contains
Node
+id: string
+label: string
+type: string
+connections: NodeConnection[]
+depth?: number
+score?: number
contains
NodeConnection
+targetId: string
+relation: string
Frontend Services
«singleton» apiClient
instance of
APIClient
-baseUrl: string
+askQuestion(request: QuestionRequest): Promise<AnswerResponse>
+checkHealth(): Promise<HealthResponse>
+initHublink(): Promise<any>
+listConversations(): Promise<Conversation[]>
+getConversation(id: string): Promise<any>
+createConversation(title?: string): Promise<Conversation>
+deleteConversation(id: string): Promise<void>
+deleteAllConversations(): Promise<void>
+persistQaExchange(conversationId, question, answer, nodes, sources, graphMaxDepth): Promise<void>
returns
uses
returns
AnswerResponse
+answer: string
+nodes: Node[]
+retrievalMode: string
+sources: string[]
+messageId: string
+maxDepth?: number
QuestionRequest
+question: string
+retrievalMode: RetrievalMode
+llmModel: string
+numberOfHubs: number
+topicEntityId?: string
+conversationId?: string
HealthResponse
+status: string
+hublinkAvailable: boolean
+message: string
Backend sends AnswerResponseSchema
Frontend receives AnswerResponse
(Same structure, different languages)
Frontend sends QuestionRequest
Backend receives QuestionRequestSchema
(Same structure, different languages)
HTTP GET /api/v1/graph/node-neighbors
HTTP GET/POST/DELETE /api/v1/conversations/
HTTP POST /api/v1/qa/ask
HTTP GET /api/v1/qa/health
HTTP POST /api/v1/qa/init
Backend
startup init
Core Config
«BaseSettings» Settings
+api_host: str
+api_port: int
+debug: bool
+orkg_email: Optional[str]
+orkg_password: Optional[str]
+vdl_api_key: Optional[str]
+retrieval_kg_config_path: str
+retrieval_llm_configs_path: str
+enable_rate_limit: bool
+rate_limit_requests: int
API
«FastAPI» FastAPIApp
+lifespan(app): AsyncContextManager
uses
includes
«Middleware» CORSMiddleware
«APIRouter» APIv1Router
+prefix: str = "/api/v1"
includes
includes
includes
«APIRouter» ConversationsRouter
+prefix: str = "/conversations"
+list_conversations(): List[ConversationSchema]
+create_conversation(body): ConversationSchema
+get_conversation(id: str): ConversationDetailSchema
+delete_conversation(id: str): 204
+delete_all_conversations(): 204
«APIRouter» GraphExploreRouter
+prefix: str = "/graph"
+get_node_neighbors(node_id: str): NodeNeighborsResponse
«APIRouter» QARouter
+prefix: str = "/qa"
+init_hublink(): Dict
+ask_question(request): AnswerResponseSchema
+health_check(): Dict
receives
returns
uses
uses
persists exchange
uses
Conversations Module
ConversationService
-_repo: ConversationRepository
+list_conversations(): List[ConversationSchema]
+create_conversation(title?): ConversationSchema
+get_conversation_detail(id): Optional[ConversationDetailSchema]
+delete_conversation(id): bool
+delete_all_conversations(): int
+persist_qa_exchange(conversation_id, question, answer, nodes, sources, graph_max_depth): void
uses
gets graph from
ConversationRepository
-_db: AsyncSession
+list_conversations(): List[ConversationModel]
+create_conversation(title): ConversationModel
+get_conversation_with_messages(id): Optional[ConversationModel]
+delete_conversation(id): bool
+delete_all_conversations(): int
+add_messages(conversation_id, user_content, assistant_content, ...): void
persists
«SQLAlchemy» ConversationModel
+created_at: DateTime
+updated_at: DateTime
+messages: List[MessageModel]
+id: String(36)
+title: String(255)
contains
«SQLAlchemy» MessageModel
+content: Text
+nodes: JSON
+sources: JSON
+graph_max_depth: Integer
+created_at: DateTime
+id: String(36)
+conversation_id: String(36)
+role: String(16)
Graph Load Module
GraphLoadService
+graph: Optional[Any]
-_available: bool
+is_available(): bool
uses
GraphLoader
+load_graph(config): Any
-_hydrate_orkg_cache_tables_from_json(graph, path): void
-_get_orkg_cache_path(kg_config): str
Graph Explore Module
GraphExploreService
-_explorer: GraphExplorer | None
+get_node_neighbors(node_id: str): List[GraphNode]
+is_available(): bool
uses
GraphExplorer
-_graph: Any
-_entities_by_id: Dict
-_incident_triples_by_node_id: Dict
-_is_index_ready: bool
-_index_lock: RLock
+get_neighbor_nodes(node_id: str): List[GraphNode]
-_ensure_index(): void
uses
«module» NodeBuilder
+build_answer_nodes_sources(retrieval_answer): Tuple[str, List[GraphNode], List[str]]
+build_graph_node(entity, score): GraphNode
+build_graph_nodes_from_edges(edges): List[GraphNode]
-_contexts_to_graph_nodes(contexts): List[GraphNode]
-_assign_graph_depths(nodes_by_id): void
Guardrails Module
GuardrailsService
-available: bool
-input_guard: Optional[Guard]
-output_guard: Optional[Guard]
+validate_input(question: str): Tuple[bool, str]
+validate_output(answer: str): Tuple[str, bool, str]
+is_available(): bool
QA Module
RetrievalService
-hublink_service: HubLinkService
-guardrails_service: GuardrailsService
+ask(question, retrieval_mode, llm_model, number_of_hubs, topic_entity_id): RetrievalResult
+get_status(): Dict
uses
uses
HubLinkService
-hublink_available: bool
-retriever: Optional[HubLinkRetrieverForUser]
-graph: Optional[KnowledgeGraph]
-config_loader: HublinkConfigLoader
+query(question, retrieval_mode, llm_model, number_of_hubs, topic_entity_id): Tuple
+is_available(): bool
+get_status(): Dict
implements
uses
uses
uses
«interface» IHubLinkService
+query(question, retrieval_mode, llm_model, number_of_hubs, topic_entity_id): Tuple
+is_available(): bool
HublinkConfigLoader
+load_kg_retrieval_config(): KGRetrievalConfig
SetupManager
+secret_manager: SecretManager
+settings: Settings
+set_up_orkg_credentials(): void
+set_up_vdl_api_key(): void
Schemas (Pydantic)
«BaseModel» ConversationDetailSchema
+id: str
+title: str
+created_at: datetime
+messages: List[MessageSchema]
contains
«BaseModel» MessageSchema
+id: str
+conversation_id: str
+role: str
+content: str
+nodes: Optional[list]
+sources: Optional[list]
+graph_max_depth: Optional[int]
«BaseModel» NodeNeighborsResponse
+nodes: List[GraphNode]
contains
«BaseModel» AnswerResponseSchema
+answer: str
+nodes: List[GraphNode]
+retrieval_mode: str
+sources: List[str]
+message_id: str
+max_depth: Optional[int]
+guardrails_warning: Optional[str]
contains
«BaseModel» GraphNode
+id: str
+label: str
+type: str
+connections: List[NodeConnectionSchema]
+depth: Optional[int]
+score: Optional[float]
contains
«BaseModel» NodeConnectionSchema
+target_id: str
+relation: str
«BaseModel» QuestionRequestSchema
+question: str
+retrieval_mode: Literal['direct', 'graph']
+llm_model: str
+number_of_hubs: int
+topic_entity_id: Optional[str]
+conversation_id: Optional[str]
«BaseModel» ConversationSchema
+id: str
+title: str
+created_at: datetime
+updated_at: datetime
«BaseModel» HealthResponseSchema
+status: str
+hublink_available: bool
+message: str
«BaseModel» ErrorResponse
+error: str
+detail: Optional[str]
+fallback_used: bool
uses
uses
uses
creates
reads cache from
creates/loads
External (sqa_system)
KGRetrievalConfig
+hublink_params: Dict
+llm_config: Dict
+embedding_config: Dict
SecretManager
HubLinkRetrieverForUser
-settings: HubLinkSettings
+retrieve_knowledge(query_text, topic_entity_id, topic_entity_value): RetrievalAnswer
+build_index(root_entity_types, root_entity_ids): void
-_update_retriever_settings(number_of_hubs, retrieval_mode): void
-_update_llm_adapter(llm_model): void
uses
KnowledgeGraphManager
+get_item(config: KnowledgeGraphConfig): KnowledgeGraph
uses
KnowledgeGraphFactoryRegistry
+get_factory_class(graph_type: str): type
resolves
ORKGKnowledgeGraphFactory
+create(config: KnowledgeGraphConfig): ORKGRemoteGraph
creates
ORKGRemoteGraph
-base_url: str
-subgraph_root_entity_id: str
-cache_manager: CacheManager
+cache_subgraph_key: str
+cache_publication_roots_key: str
+validate_graph_connection(): bool
+get_entity_by_id(entity_id): Knowledge
+cache_subgraph(): void
uses
extends
«Singleton» CacheManager
-db_path: str
+add_data(meta_key, dict_key, value): void
+get_data(meta_key, dict_key): Optional[Any]
+get_table(meta_key): Optional[dict]
+clear_cache(): void
+get_hash_value(text: str): str
KnowledgeGraph
#config: KnowledgeGraphConfig
VDLEmbeddingAdapter
+VDL_BASE_URL: str
+prepare(): void
+validate(): ValidationResult
extends
VDLLLMAdapter
+VDL_BASE_URL: str
+prepare(): void
+validate(): ValidationResult
extends
creates
LLMProvider
uses
creates
uses
EmbeddingAdapter
#embedding_config: EmbeddingConfig
+validate(): ValidationResult
+prepare(): void
LLMAdapter
#llm_config: LLMConfig
#llm: BaseChatModel
+generate(prompt: str): Any
+validate(): ValidationResult
+prepare(): void
EndpointEnvVariable
HUGGINGFACE_API_TOKEN
OPENAI_API_KEY
GOOGLE_API_KEY
VDL_API_KEY
EndpointType
HUGGINGFACE
OPENAI
OLLAMA
GOOGLEAI
VDL